

Technical Assessment

Debugging

Customers are reporting that calculations for Year-Over-Year (YoY) spending are inaccurate. Please refactor the following function so that it passes the provided unit tests

```
def calculate_yoy_change(spending_data):
    sorted_years = sorted(spending_data.keys())
    changes = {}

    for i in range(0, len(sorted_years)):
        current_year = sorted_years[i]
        previous_year = sorted_years[i - 1]
        current_spending = spending_data[current_year]
        previous_spending = spending_data[previous_year]

        change = (current_spending - previous_spending) /
previous_spending * 100
        changes[current_year] = change

    return changes

import unittest

class TestCalculateYoYChange(unittest.TestCase):

    def test_yoy_positive_increase(self):
        spending_data = {2020: 1000, 2021: 1200, 2022: 1300, 2023:
1600}
        expected = {2021: 20.0, 2022: 8.333333333333332, 2023:
23.076923076923077}
        self.assertEqual(calculate_yoy_change(spending_data),
expected)

    def test_yoy_with_zero_previous(self):
        spending_data = {2020: 0, 2021: 500}
        expected = {2021: None}
        self.assertEqual(calculate_yoy_change(spending_data),
expected)

    def test_yoy_negative_spending(self):
        spending_data = {2020: 1000, 2021: 1200, 2022: -100, 2023:
1600}
        expected = {
            2021: 20.0,
            2022: -108.33333333333333,
```

```

        2023: 1700.0
    }
    self.assertEqual(calculate_yoy_change(spending_data),
expected)

# Run the tests
unittest.TextTestRunner().run(unittest.makeSuite(TestCalculateYoYChange))

FFE
=====
ERROR: test_yoy_with_zero_previous (__main__.TestCalculateYoYChange)
-----
Traceback (most recent call last):
  File
"/var/folders/vg/b748kyp1lxz07hq89w10p__00000gn/T/ipykernel_51490/1059
704487.py", line 13, in test_yoy_with_zero_previous
    self.assertEqual(calculate_yoy_change(spending_data), expected)
  File
"/var/folders/vg/b748kyp1lxz07hq89w10p__00000gn/T/ipykernel_51490/1322
337770.py", line 11, in calculate_yoy_change
    change = (current_spending - previous_spending) /
previous_spending * 100
ZeroDivisionError: division by zero

=====
FAIL: test_yoy_negative_spending (__main__.TestCalculateYoYChange)
-----
Traceback (most recent call last):
  File
"/var/folders/vg/b748kyp1lxz07hq89w10p__00000gn/T/ipykernel_51490/1059
704487.py", line 22, in test_yoy_negative_spending
    self.assertEqual(calculate_yoy_change(spending_data), expected)
AssertionError: {2020: -37.5, 2021: 20.0, 2022: -108.33333333333333,
2023: -1700.0} != {2021: 20.0, 2022: -108.33333333333333, 2023:
1700.0}
- {2020: -37.5, 2021: 20.0, 2022: -108.33333333333333, 2023: -1700.0}
? -----
+ {2021: 20.0, 2022: -108.33333333333333, 2023: 1700.0}

=====
FAIL: test_yoy_positive_increase (__main__.TestCalculateYoYChange)
-----
Traceback (most recent call last):
  File
"/var/folders/vg/b748kyp1lxz07hq89w10p__00000gn/T/ipykernel_51490/1059
704487.py", line 8, in test_yoy_positive_increase
    self.assertEqual(calculate_yoy_change(spending_data), expected)
AssertionError: {2020: -37.5, 2021: 20.0, 2022: 8.333333333333332,

```

```
2023: 23.076923076923077} != {2021: 20.0, 2022: 8.333333333333332,
2023: 23.076923076923077}
- {2020: -37.5, 2021: 20.0, 2022: 8.333333333333332, 2023:
23.076923076923077}
? -----
```

```
+ {2021: 20.0, 2022: 8.333333333333332, 2023: 23.076923076923077}
```

```
-----
Ran 3 tests in 0.008s
```

```
FAILED (failures=2, errors=1)
```

```
<unittest.runner.TextTestResult run=3 errors=1 failures=2>
```

SQL Knowledge

You are given a database table named **tickets** with the columns **ticket_id**, **status**, **priority**, and **assigned_to**. Write a SQL query to find all open tickets (where status is 'open') assigned to 'engineer1' with a priority of 'high'

Python Coding

Task: Write a function that takes in a list of sales transactions, where each transaction is represented as a dictionary containing a product name and quantity sold. The function should return a dictionary that aggregates the total quantity sold per product.

Example Input:

```
transactions = [
    {"product": "apple", "quantity": 10},
    {"product": "banana", "quantity": 5},
    {"product": "apple", "quantity": 3},
    {"product": "orange", "quantity": 8},
    {"product": "banana", "quantity": 2}
]
```

Expected Output:

```
{
    "apple": 13,
    "banana": 7,
    "orange": 8
}
```